

List Comprehension

1. What is a List?

In Python, a list is a **collection of items**. Lists are written using square brackets `[]`.

Example:

```
Python
numbers = [1, 2, 3, 4]
names = ["Alice", "Bob"]
mixed = [1, "hello", True, 3.5]
```

2. What is a List Comprehension?

A **list comprehension** is a short and powerful way to create new lists by using a loop in one line.

3. List Comprehension Syntax

```
Python
[expression for item in iterable if condition]
```

Explanation:

- **expression**: what to do with the item (e.g., transform or return it)
- **item**: the variable name for each value in the iterable
- **iterable**: the collection you're looping over (e.g., a list)
- **condition** (optional): filter to include only matching items

4. Basic Examples

a. Square numbers in a range:

```
Python
squares = [x * x for x in range(6)]
print(squares)
# Output: [0, 1, 4, 9, 16, 25]
```

b. Filter even numbers:

```
Python
numbers = [1, 2, 3, 4, 5, 6]
evens = [n for n in numbers if n % 2 == 0]
print(evens)
# Output: [2, 4, 6]
```

c. Uppercase a list of strings:

```
Python
names = ["alice", "bob", "charlie"]
upper_names = [name.upper() for name in names]
print(upper_names)
# Output: ['ALICE', 'BOB', 'CHARLIE']
```

5. Create Lists and Apply List Comprehensions

Let's create custom lists and do operations.

List 1: Product Names (strings)

```
Python
products = ["laptop", "phone", "tablet", "monitor"]
```

Make all names uppercase

```
Python
upper_products = [p.upper() for p in products]
# ['LAPTOP', 'PHONE', 'TABLET', 'MONITOR']
```

List 2: Prices (numbers)

```
Python
prices = [1000, 800, 450, 300]
```

Apply 10% discount

```
Python
discounted = [price * 0.9 for price in prices]
# [900.0, 720.0, 405.0, 270.0]
```

List 3: Stock status (boolean)

```
Python
in_stock = [True, False, True, False]
```

Get indexes of products in stock

```
Python
available = [i for i, stock in enumerate(in_stock) if stock]
# [0, 2]
```

List 4: Product info as tuples

```
Python
product_info = [("Laptop", 1000), ("Phone", 800), ("Tablet",
450)]
```

Get names of products with price over 700

```
Python
expensive = [name for name, price in product_info if price >
700]
# ['Laptop', 'Phone']
```

List 5: List of dictionaries

```
Python
products_data = [
    {"name": "Laptop", "price": 1000, "stock": 3},
    {"name": "Phone", "price": 800, "stock": 0},
    {"name": "Tablet", "price": 450, "stock": 5}
]
```

Get available product names

```
Python
available_names = [p["name"] for p in products_data if
p["stock"] > 0]
# ['Laptop', 'Tablet']
```

Get discounted prices for products in stock

```
Python
discounted_products = [{p["name"]: p["price"] * 0.9} for p in
products_data if p["stock"] > 0]
# [{'Laptop': 900.0}, {'Tablet': 405.0}]
```

6. List Comprehension with All Data Types

Data Type	Example	List Comprehension Example
String	"apple"	<code>[s.upper() for s in fruits]</code>
Integer	1200	<code>[x + 100 for x in prices]</code>
Float	99.99	<code>[round(p * 1.1, 2) for p in prices]</code>
Boolean	True / False	<code>[i for i, b in enumerate(in_stock) if b]</code>
Tuple	("Phone", 800)	<code>[name for name, price in products if price > 700]</code>
Dictionary	<code>{"name": "Phone", "price": 800}</code>	<code>[p["name"] for p in products if p["stock"] > 0]</code>

7. Nested List Comprehension

Example: Flatten a list of product color options

```
Python
products_colors = [
    {"name": "Laptop", "colors": ["Silver", "Black"]},
    {"name": "Phone", "colors": ["Gold", "Blue"]}
]

all_colors = [color for product in products_colors for color
in product["colors"]]
# ['Silver', 'Black', 'Gold', 'Blue']
```

Task: Show available product names with discounted prices

Python

```
products_data = [  
    {"name": "Laptop", "price": 1000, "stock": 3},  
    {"name": "Phone", "price": 800, "stock": 0},  
    {"name": "Tablet", "price": 450, "stock": 5}  
]  
  
result = [f"{p['name']} - ${p['price'] * 0.9:.2f}" for p in  
products_data if p["stock"] > 0]  
# ['Laptop - $900.00', 'Tablet - $405.00']
```